

Final Project: A Complete Testing Effort

Group project, applying the whole course

Group size	Up to four students per group
Deadline	Sunday, 13 September 2026, end of day
Submission	One submission per group on Google Classroom: a repository link and the documents
Weight	Course final project
Instructor	Abel Tadesse

1. Overview

In this project your group will build a small application and carry out a complete, professional testing effort on it, applying every technique from this course. The application is only the vehicle; the testing is the point. You will plan the testing, design test cases with formal techniques, automate tests at every level, run them in continuous integration with both GitHub Actions and Jenkins, manage defects, measure quality, and report your findings. The concept coverage map in section 8 shows exactly where each course topic must appear.

2. The application to build

Build a small web application with both business logic and a user interface. It does not need many features; it needs the right shape so that every testing technique has something to work on. Your application must include:

- Inputs with ranges or categories, such as an age, a price band or a grade, so equivalence partitioning and boundary value analysis apply.
- Logic that depends on a combination of conditions, such as a discount or eligibility rule, so a decision table applies.
- A stateful workflow, such as an order or a booking moving between statuses, so state transition testing applies.
- A multi-step user journey through the interface, such as log in then place an order, so system tests with Selenium apply.

Choose one of these domains, or propose your own of similar size:

- A library system: members, borrowing and returning, fines by days overdue.
- An online shop: a cart, a discount rule, and an order that moves from placed to shipped.
- A clinic booking system: appointments that are requested, confirmed, attended or cancelled.
- A student results system: marks entered, grades derived by band, and results published.

Keep the technology simple and testable. Any stack is allowed, provided it supports unit testing and a web interface Selenium can drive. Two well-supported choices:

Java: Spring Boot or plain Java, JUnit 5, Selenium, Maven Python: Flask or FastAPI, pytest, Selenium, pip
--

3. What you must do

The work is organised into parts. Each part maps to units of the course. Do them roughly in order, since later parts build on earlier ones.

Part A. Test plan

Write a test plan for your project. It must state the scope, the approach (which levels and techniques you will use), the entry and exit criteria for testing, a risk-based prioritisation of what to test hardest, a short schedule, and the roles of each group member.

Part B. Test design

Produce a test design document that derives test cases using formal techniques. You must apply, and clearly show your working for, equivalence partitioning, boundary value analysis, a decision table, and state transition testing. For each technique, identify the partitions, boundaries, rules or transitions, then list the test cases they produce.

Part C. Automated tests at every level

Implement automated tests, organised by level and following the test pyramid: many fast unit tests, fewer integration tests, and a small number of system tests.

- Unit tests for your business logic, including at least one test double (a stub, fake, mock or spy) to isolate a unit from a collaborator.
- Integration tests that check two or more components working together.
- System tests that drive the application through its interface using Selenium, organised with the Page Object pattern.

Part D. Coverage

Measure code coverage of your test suite and report it. Aim for at least 80 percent branch coverage of your core business logic. Explain any code you deliberately left uncovered.

Part E. Continuous integration and regression

Automate your suite so it runs on every change.

- A GitHub Actions pipeline that installs dependencies, runs all tests, and reports coverage on every push.
- A Jenkins pipeline defined in a Jenkinsfile, run using the Docker setup from the Jenkins lab.
- A demonstrated regression: introduce a change that breaks a test, show the pipeline catching it, then fix it. Record this in your report with the failing and passing build.

Part F. Defect management

Keep a defect log for defects you find during the project. Each entry needs an identifier, steps to reproduce, expected and actual results, a severity and a priority, and a status tracked through the defect lifecycle from new to closed.

Part G. Metrics

Compute and interpret a small set of metrics for your project: defect density, defect removal efficiency, the count of defects found versus any that escaped your tests, and your coverage figure. Present them in a table with one or two sentences of interpretation each.

Part H. Test summary report

Write a test summary report: what was tested and what was not, the results against your exit criteria, the outstanding defects and the residual risk, and a clear recommendation on whether the application is ready to release.

Part I. Foundations reflection

Include a short reflection, half a page, that takes one real defect from your project and explains it in terms of error, fault and failure, and states whether it was caught by verification or by validation.

4. Deliverables

Submit one repository and a set of documents. The repository must contain:

- The application source code.
- The automated test suites: unit, integration and system tests.
- The CI configuration: a GitHub Actions workflow and a Jenkinsfile with its Docker setup.
- A README explaining how to run the application, the tests and the pipelines.

Alongside the repository, submit these documents, as PDF:

- The test plan (Part A).
- The test design document (Part B).
- The defect log and metrics (Parts F and G).
- The test summary report and foundations reflection (Parts H and I).

5. Submission

Submit once per group on Google Classroom by the deadline. Include every group member's name and student identifier on the first page of each document and in the README. Share the repository so the instructor can access it, and check that your pipelines are visible. Late submissions are handled under the course late policy.

6. Assessment

The project is marked out of 100 as follows. Marks reward evidence that each concept was applied correctly, not merely mentioned.

Criterion	Marks
Test plan: scope, approach, entry and exit criteria, risk-based prioritisation	10
Test design: equivalence partitioning, boundary value analysis, decision table, state transition, documented with derived cases	15
Unit and integration tests, using at least one test double, following the pyramid	15
System tests with Selenium and the Page Object pattern	15
Code coverage: target met on core logic, and reported	8
Continuous integration: a GitHub Actions pipeline, plus a demonstrated regression caught by it	12
Jenkins: a working pipeline via the provided Docker setup	8
Defect log and metrics: severity and priority, the defect lifecycle, defect density and removal efficiency	10
Test summary report and the foundations reflection	7

Total: 100 => 20% of Final Grade

7. Group work and academic integrity

Work in your group of up to four. Every member must contribute, and contributions should be visible in the repository history. You may be asked to submit a short peer contribution statement. You may discuss ideas with other groups, but each group's application, tests and documents must be its own work. Plagiarism and the unauthorised sharing of solutions are serious academic offences.

Use of AI tools: you may use them to learn and to help write code, but you must understand and be able to explain everything you submit, and you remain responsible for its correctness.

8. Concept coverage map

Every concept below must appear in your project. Use this as a checklist, and cite where each is demonstrated in your submission.

Concept	From	Where it appears in your project
Error, fault, failure; verification vs validation	Unit 1	Foundations reflection (Part I) and defect log

Test design techniques	Unit 2	Test design document (Part B)
Coverage: statement and branch	Unit 2, 4	Coverage report in CI (Part D)
Levels: unit, integration, system, acceptance	Unit 3	Automated suites (Part C) and UAT in the plan
Test doubles: stub, mock, fake, spy	Unit 3	Unit tests (Part C)
The test pyramid	Unit 3, 4	Balance of the suites (Part C)
Selenium and the Page Object pattern	Unit 3, tutorial	System tests (Part C)
Regression testing and automation	Unit 4	CI pipeline and regression demo (Part E)
Continuous integration	Unit 4, Jenkins	GitHub Actions and Jenkins (Part E)
Test management: plan, entry and exit criteria, risk	Unit 5	Test plan (Part A)

Concept	From	Where it appears in your project
Defect management and metrics	Unit 5	Defect log and metrics (Parts F, G)
Quality and the stopping decision	Unit 5	Test summary report (Part H)

9. Suggested timeline

You have about three weeks. A workable plan:

- Week 1: agree the application and build a first working version. Write the test plan and the test design document.
- Week 2: implement unit, integration and Selenium tests. Set up GitHub Actions and Jenkins. Reach your coverage target.
- Week 3: run the regression demonstration, keep the defect log, compute the metrics, write the summary report, and tidy the repository.

Start the continuous integration setup early. Getting the first green pipeline running often takes longer than expected, and everything else is easier once it works.